INTERVIEW PREPARATION

KPMG — SRE / Platform Engineering

Payroll via Prism HRC · Second Round (Virtual) · Final Round: F2F

Candidate: Saroj Kumar Nayak

Role focus: Software / Platform / Site Reliability Engineering

Core JD themes: 10+ yrs SRE/Platform experience · Go/Python/Java/Ruby/Perl · Infrastructure as Code · Cloud Fundamentals · Linux Administration & Troubleshooting · SLA/SLO Understanding

Desirable: AWS · Kubernetes · Networking · Security · Storage · Observability (Prometheus/Graphite/Grafana/Kibana/Elasticsearch)

Document: 126 questions across 13 topic sections

Prepared as a print-ready review companion · check each box as you master it

Table of Contents

Role & Positioning	8 questions
Programming (Go/Python/Java/Ruby/Perl)	10 questions
Infrastructure as Code (Terraform)	12 questions
Cloud Computing Fundamentals	8 questions
Linux Administration & Troubleshooting	12 questions
SLAs, SLOs & Error Budgets	10 questions
AWS Solutions	12 questions
Kubernetes	12 questions
Networking	8 questions
Security	8 questions
Storage	6 questions
Observability (Prometheus/Graphite/Grafana/ELK)	12 questions
Behavioral & Interpersonal	8 questions

Role & Positioning

KPMG / PRISM HRC CONTEXT

Walk me through your background.

14+ years in SRE/DevOps/Platform Engineering across banking and enterprise environments — Barclays, Flyingedge Solutions/Russell Investments, Terawe Technologies (Microsoft/Azure Synapse), Unique Vacations/Sandals International, RSAB IT, and Orange Business Services. Core strength is operating and evolving large-scale, sophisticated systems: AWS (6+ yrs), Kubernetes (6+ yrs), Terraform (3-6 yrs), plus Azure depth (AZ-104, SC-300, SC-100) that most AWS-only SREs don't have. Anchor with one real number, e.g. a ~35% MTTD reduction from an observability overhaul at Terawe, or a ~60% reduction in manual deployment effort from an automation initiative — then say you're targeting senior/AVP-level Platform Engineering and SRE roles where that breadth compounds.

What do you know about KPMG and this role?

KPMG is one of the Big Four global professional-services firms — audit, tax, and advisory — with a large internal technology and consulting arm that builds and runs platforms for both internal use and client engagements. An SRE/Platform role here likely means supporting internal enterprise platforms and/or client-facing delivery infrastructure with strict reliability, security, and audit expectations (consistent with a Big Four's compliance posture). Frame your consulting-adjacent Barclays/Russell Investments experience as directly relevant — you already know how to operate reliably inside a regulated, audited environment.

This role is payrolled via Prism HRC rather than KPMG directly — are you comfortable with that arrangement?

Bring your own: state your comfort level plainly (most contractors are fine with a staffing/payroll vendor as long as the day-to-day work, reporting line, and project are genuinely with the client). It's reasonable to ask Prism HRC or the KPMG hiring manager to clarify contract duration, extension/conversion-to-FTE possibility, and who owns performance reviews — asking this shows diligence, not hesitation.

You have 14+ years; the JD asks for 10+. How do you frame the extra experience without sounding overqualified?

Position it as depth and range rather than just tenure: dual-cloud (AWS+Azure), full lifecycle from build to steady-state operations, and exposure to regulated/banking environments the JD doesn't explicitly ask for but almost certainly values given KPMG's client base. Tie it to what you want next — broader platform ownership and mentoring scope — rather than implying the role itself is too small.

Why are you looking to move right now?

Bring your own — keep it forward-looking (seeking broader platform-ownership scope, dual-cloud environments, or a step toward AVP/Architect-level responsibility) rather than negative about a current/previous employer.

This is a second, virtual round with a final F2F round still to come — what do you think this round is testing for?

Likely deeper technical validation of what a resume/first-round screen already flagged (Terraform, K8s, AWS depth, Linux troubleshooting, SLO literacy) plus a check on communication clarity and team fit, since the JD explicitly calls out interpersonal skills and English proficiency. Treat this round as: prove the depth is real, and prove you can explain it clearly to a mixed technical/non-technical panel.

Where do you see yourself in your SRE/Platform Engineering career path?

Bring your own — but the natural framing given your background is: senior IC or AVP/Architect-track Platform Engineer, owning reliability and platform standards across multiple teams rather than a single service, with growing mentorship responsibility.

The JD frames this broadly as 'software, platform, or site reliability engineering' — how do you read that?

It signals the hiring team cares about the underlying skill set (building and operating scalable, reliable systems) more than a rigid title. Software engineers who build resilient services, platform engineers who build the internal tooling other teams run on, and SREs who apply engineering discipline to operations are treated as overlapping candidate pools for this seat — so you should speak to all three angles: you write/automate, you've built platform tooling, and you own reliability outcomes.

Programming (Go/Python/Java/Ruby/Perl)

LANGUAGE FIT & FUNDAMENTALS

☐Which of Go, Python, Java, Ruby, or Perl are you strongest in, and why does that matter for this role?

Lead with Python (and Bash) as your primary automation/scripting language — it's the most common SRE glue language for AWS APIs (boto3), infra automation, and log/metric processing. If Java or another comes up from your Terawe/Barclays work, mention it as secondary. The point for the interviewer isn't which language — it's that you can read, write, and safely modify production automation code, which you can back with real examples (deployment automation, alert tooling).

☐The team may use Go primarily — how would you approach ramping up if you don't have deep Go experience?

Be honest rather than overclaiming: strong general programming fundamentals (typing, concurrency concepts, error handling) transfer, and Go's simplicity (no classes, explicit error returns, single binary output) makes it fast to become productive-enough for SRE tooling within weeks. Mention you'd start by reading and modifying existing Go tools/exporters before writing net-new services, which is the realistic on-ramp for most SREs picking up Go.

☐Describe a real automation script or tool you've built and the impact it had.

Bring your own, anchored to a verified result — e.g. an automation initiative that cut manual deployment effort by roughly 60%, or tooling that contributed to a ~40% reduction in alert noise. Structure it as: manual/painful process → what you automated and how (language, trigger, safeguards) → measured outcome.

☐What's the difference between a one-off script and production-grade automation?

Production-grade automation has: idempotency (safe to re-run), explicit error handling and logging, no hardcoded secrets/credentials, a dry-run or plan mode where possible, tests (even basic ones), and a rollback path. A one-off script usually skips all of that because it's meant to run once, by you, and be thrown away.

☐How do you handle secrets inside automation scripts?

Never hardcode or commit secrets. Pull from a secrets manager (AWS Secrets Manager/Parameter Store, HashiCorp Vault, Azure Key Vault) or environment injection at runtime via the CI/CD system's own secret store, with least-privilege IAM scoping the script's own credentials to only what it needs.

☐What does idempotency mean in automation, and why does it matter operationally?

An idempotent operation produces the same end state no matter how many times it's run — critical because retries after partial failures, timeouts, or re-triggered pipelines are routine in production. A non-idempotent deploy script that re-runs after a timeout can double-create resources or corrupt state; an idempotent one just confirms the desired state and exits cleanly.

☐Sketch how you'd write a Python script using boto3 to find and remediate unhealthy EC2 instances behind a load balancer.

At a high level: use boto3's elbv2 client to list target health, filter for unhealthy targets, cross-reference with the ec2 client to check instance status checks, log findings, and either reboot/replace via an Auto Scaling Group action (preferred over a direct terminate, so the ASG handles safe replacement) — wrapped in a dry-run flag and Slack/SNS notification so it's observable, not a silent auto-fix.

☐How do you approach code review for infrastructure/automation code specifically?

Beyond normal readability, check for: blast radius (what happens if this runs against prod by mistake), idempotency, credential handling, and whether it has a rollback or dry-run path. For Terraform/IaC changes specifically, always review the plan output, not just the diff.

☐How do you test infrastructure automation before it touches production?

Layered: unit tests for pure logic, a dry-run/plan mode against a real environment, then a staging/non-prod apply with the same pipeline path production will use, before promoting the exact same artifact to prod — never re-writing the change by hand for each environment.

☐Give an example of a bug in an automation script that could have caused an outage, and how you'd guard against that class of bug in review.

Bring your own if you have a real one — otherwise speak generally: e.g. a script that assumed a single result from an API call and broke (or acted on the wrong resource) when the account had multiple matches. Guard with explicit assertions on expected result count, and fail loudly rather than silently proceeding on unexpected input.

Infrastructure as Code (Terraform)

CORE CONCEPTS & GOTCHAS

☐What is Infrastructure as Code and why does it matter for reliability specifically?

IaC means infrastructure is defined in versioned, reviewable code rather than made through manual console changes. For reliability it matters because it makes environments reproducible (disaster recovery becomes 'run the code' rather than 'remember what we clicked'), changes are peer-reviewed before they hit production, and drift between environments becomes detectable rather than a mystery during an incident.

☐What is Terraform state, and why do remote backend + state locking matter?

State is Terraform's record of what it believes exists and how config maps to real resource IDs. A remote backend (e.g. S3 + DynamoDB, or Terraform Cloud) makes state shared and durable instead of a single laptop's local file; state locking prevents two people or two CI runs from applying concurrently and corrupting state or double-creating resources.

☐Why is for_each generally preferred over count for managing multiple similar resources?

count indexes resources by position (0,1,2...), so removing or reordering an item in the middle shifts every subsequent index — Terraform then wants to destroy and recreate resources that didn't actually change. for_each keys resources by a stable value (like a map key or set member), so removing one item only affects that one resource.

☐If you mark a variable sensitive = true in Terraform, is the value actually protected?

Not fully — sensitive = true hides the value from CLI output/plan display, but it is still stored in plaintext in the state file by default. True protection requires state encryption (e.g. an encrypted S3 backend) and/or keeping the actual secret out of Terraform entirely (reference a secrets manager ARN instead of the raw value). Newer Terraform versions (1.10/1.11) introduced ephemeral and write-only attributes specifically to avoid persisting sensitive values to state at all.

☐How do you detect and handle configuration drift?

terraform plan against the current state will show drift as a diff even without code changes, since it refreshes against real infrastructure. Handle it by either updating code to match an intentional manual change (then re-applying) or reverting the manual change to match code — but the real fix is process: restrict console/CLI write access so drift becomes rare, not something you're constantly reconciling.

☐How do you design reusable Terraform modules across multiple environments?

Keep modules environment-agnostic — parameterize via variables (instance size, count, tags) rather than hardcoding environment specifics, and keep environment-specific values in separate .tfvars files or a thin per-environment root module that calls the shared module. Avoid embedding conditionals for 'if prod do X' inside the module itself; that logic belongs at the calling layer.

☐Why is the -target flag risky, and when is it acceptable to use?

-target applies to a specific resource and its dependencies only, bypassing Terraform's full-graph reconciliation — so it can silently leave other pending changes unapplied or cause state to diverge from what a full plan would show. Acceptable as a targeted emergency fix or to unblock a specific broken resource, but should always be followed by a full plan/apply to confirm the overall state is consistent.

☐How does terraform import work, and what's a common mistake with it?

import binds an existing real resource to a Terraform state address so Terraform starts managing it — but import does NOT generate the matching HCL configuration for you. The common mistake is importing into state without writing accurate matching config first; the next plan will then show a large, confusing diff as Terraform tries to reconcile config against the real resource.

☐What is the moved block for, and why is it considered high-value to know?

moved lets you refactor Terraform code (rename a resource, move it into a module) while telling Terraform 'this is the same resource, just relocated' — avoiding a destroy/recreate. It's high-value because the failure mode without it is silent and expensive: a refactor that looks cosmetic in a PR can actually queue up a destroy-and-recreate of a production resource (like a database) if Terraform can't tell it's the same object.

☐How would you manage multiple environments (dev/stage/prod) — workspaces, separate state per environment, or Terragrunt?

Bring your own preference, but the generally safer pattern at scale is separate state files per environment (via separate directories or a tool like Terragrunt) rather than Terraform workspaces sharing one backend/config — workspaces make it easy to accidentally apply against the wrong environment since the code path is identical and only a workspace name differs.

☐How do you roll back a bad terraform apply?

Terraform doesn't have a built-in 'undo' — the standard approach is to revert the code change in version control and re-apply, which reconciles infrastructure back toward the previous desired state. For state-only corruption (not actual resource damage), a state backup/versioned S3 backend lets you restore a prior state file if needed.

How do you integrate Terraform into CI/CD safely?

Run terraform plan automatically on every pull request (posted as a PR comment for reviewers), require human approval before apply, restrict who/what can run apply against production (often a separate protected pipeline stage), and never let developers run terraform apply from their laptops against shared environments.

Cloud Computing Fundamentals

FOUNDATIONS

☐ Explain the shared responsibility model.

The cloud provider secures 'the cloud' (physical infrastructure, hypervisor, managed-service internals); the customer secures 'in the cloud' (data, IAM configuration, network rules, OS patching on unmanaged instances, application security). The exact line shifts depending on the service tier — e.g. AWS patches the OS for a managed RDS instance but not for a raw EC2 instance.

☐ Compare IaaS, PaaS, and SaaS.

IaaS (e.g. EC2, Azure VMs) gives raw compute/storage/network — you manage the OS up. PaaS (e.g. Elastic Beanstalk, App Service, RDS) manages the runtime/OS/patching for you — you manage code and config. SaaS (e.g. Salesforce, Office 365) is a fully managed application — you manage only your data and users.

☐ What's the difference between high availability and disaster recovery?

High availability is about minimizing downtime during normal operation or a localized failure (e.g. multi-AZ redundancy so one AZ failing doesn't take the service down). Disaster recovery is the plan for a larger-scale event (region loss, major data corruption) and is measured by RPO (how much data you can afford to lose) and RTO (how fast you must recover) — DR often involves a separate region and a documented, tested failover process.

☐ Horizontal vs vertical scaling — when do you use each?

Vertical scaling (bigger instance) is simpler but has a hard ceiling and usually requires downtime; it's fine for early-stage or single-instance stateful systems. Horizontal scaling (more instances behind a load balancer) is the standard approach for reliability at scale since it also improves fault tolerance — one instance failing doesn't take down the service — but requires the application to be stateless or externalize state.

☐ What cost-optimization levers do you use in cloud environments?

Rightsizing based on actual utilization data, Reserved Instances/Savings Plans for predictable steady-state workloads, Spot instances for fault-tolerant/stateless batch workloads, S3 lifecycle policies to move cold data to cheaper tiers, and tagging/cost-allocation to make spend visible per team so waste is caught rather than hidden in a shared bill.

☐ Multi-region vs multi-AZ — what's the tradeoff?

Multi-AZ protects against a data-center-level failure within a region with low latency and relative simplicity (most managed services support it natively). Multi-region protects against a full regional outage but adds significant complexity — data replication/consistency, DNS failover, and often materially higher cost — so it's usually reserved for services with strict regulatory or business-continuity requirements.

☐ What is the AWS/cloud Well-Architected Framework, and what are its pillars?

A structured way to evaluate a system's design against best practices, typically covering: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability. It's used as a review checklist — walking through a design and asking, for each pillar, what risks or gaps exist.

☐ How do you position your Azure certifications (AZ-104, SC-300, SC-100) for what looks like an AWS-heavy role?

Frame it directly as a strength, not a mismatch: your hands-on depth is AWS (6+ years), but the Azure certifications give you genuine dual-cloud fluency — identity/security concepts (SC-300/SC-100) and infrastructure fundamentals (AZ-104) transfer conceptually across clouds, so you ramp faster on any Azure-adjacent work KPMG's client engagements might touch, which is a real advantage in a consulting environment that likely serves clients on both platforms.

Linux Administration & Troubleshooting

DIAGNOSIS & TOOLS

Server shows high load average but CPU usage looks low — what do you check?

Load average includes processes waiting on I/O, not just CPU-bound ones — so check iostat/vmstat for disk I/O wait, and check for processes stuck in uninterruptible sleep (D state) via ps aux. High load with low CPU usually points to disk or network I/O bottlenecks, not compute.

A disk is at 100% — how do you find what's consuming space fast?

du -sh /* (or du -h --max-depth=1 on a suspect directory) to find the largest top-level directories, drilling down from there; df -h to confirm which mount is actually full. Also check for deleted-but-still-open files holding space via lsof | grep deleted, since du won't show space held by a file that's unlinked but still open by a process.

A process seems hung — how do you investigate without just killing it?

strace -p <pid> to see what syscalls it's currently making (or stuck on), lsof -p <pid> to see open files/sockets, and for a stack-level view, gdb -p <pid> with a backtrace, or jstack for a JVM process specifically. This tells you whether it's stuck on I/O, a lock, or genuinely CPU-spinning before deciding to kill it.

How does the Linux OOM killer decide which process to kill?

It scores processes by an oom_score (influenced by memory usage and an adjustable oom_score_adj), generally favoring killing the process using the most memory unless it's been explicitly protected. In containers, this often surprises people because a container can be OOMKilled by the kernel without ever logging an application-level OutOfMemoryError, if its cgroup memory limit is hit.

How do you troubleshoot a service that won't start under systemd?

systemctl status <service> for the immediate error and recent log lines, then journalctl -u <service> -n 100 --no-pager for fuller context, checking exit code and whether it's a config error, permission issue, or a dependency (After=/Requires=) that hasn't started yet.

Explain file permissions and umask with a practical example.

Permissions are read/write/execute for owner/group/other (e.g. 755 = rwxr-xr-x). umask subtracts from the default (666 for files, 777 for directories) at creation time — a umask of 022 means new files default to 644 and new directories to 755. If a deployed file unexpectedly has group-write access, checking the deploying process's umask is a common first step.

What's the difference between a zombie process and an orphan process?

A zombie has finished executing but still has an entry in the process table because its parent hasn't called wait() to read its exit status — it's dead but not yet reaped. An orphan is still running but its parent has exited before it did; orphans get re-parented to init/systemd, which will reap them normally.

Walk through your sequence for troubleshooting 'can't reach a service' end to end.

Start local and move outward: can the client resolve DNS (dig/nslookup)? Can it reach the IP at all (ping, though ICMP may be blocked)? Can it open the TCP port (nc -zv or telnet)? If that fails, check security groups/firewall rules and routing tables. If the port opens but the app doesn't respond correctly, move up to application logs and health-check behavior.

Give a real example of using awk, sed, or grep to extract something useful from a log file.

Example: grep 'ERROR' app.log | awk -F'|' '{print $3}' | sort | uniq -c | sort -rn to pull out error messages, isolate a specific field, and rank them by frequency — a fast way to spot the dominant error type during an incident without writing a script.

How do you use find and xargs together, and why combine them?

find locates files matching criteria (name, age, size), and piping to xargs lets you act on all of them in bulk efficiently — e.g. find /var/log -name '*.log' -mtime +30 | xargs rm, or find . -name '*.tmp' -print0 | xargs -0 rm to safely handle filenames with spaces.

When would you reach for lsof, and what does it tell you?

When you need to know what a process has open — files, sockets, or pipes. Classic uses: 'why can't I unmount this disk' (lsof shows what's still using it), or 'why is this port already in use' (lsof -i :PORT shows the owning process).

How do you approach a root-cause analysis using /var/log and journalctl after an incident?

Correlate timestamps across sources — start from the alert/incident time, work backward in the application log for the first anomaly, then cross-reference system logs (journalctl, /var/log/syslog or /var/log/messages) for the same window to catch OS-level events (OOM kills, disk errors, network flaps) that the application log alone wouldn't show.

SLAs, SLOs & Error Budgets

CONCEPTS & APPLICATION

☐ Explain the difference between SLA, SLO, and SLI.

SLI (Service Level Indicator) is the actual measured metric — e.g. request success rate. SLO (Service Level Objective) is your internal target for that metric — e.g. 99.9% success rate over 30 days. SLA (Service Level Agreement) is the external, often contractual, commitment to a customer — usually set looser than the internal SLO, with penalties or credits if breached, so you have margin to catch problems before they become SLA violations.

☐ How do you choose good SLIs for a service?

Pick metrics that reflect actual user experience, not just infrastructure health — request latency and success rate as experienced by the client, not just 'is the server up.' Keep the set small (a handful of golden signals) rather than trying to track everything, since too many SLIs dilutes focus during an incident.

☐ What is an error budget, and how do you use it day to day?

If your SLO is 99.9%, your error budget is the remaining 0.1% of allowed failure over the period. It's used as a decision-making tool: if the budget is healthy, teams can ship faster and take more risk; if it's nearly exhausted, the team should slow down, freeze risky releases, and focus on reliability work until it recovers.

☐ What's the difference between fast-burn and slow-burn alerting on an error budget?

Fast-burn alerts catch a severe event that would exhaust the whole budget in hours (needs immediate paging). Slow-burn alerts catch a milder, sustained degradation that would exhaust the budget over days or weeks (can often wait for business-hours attention). Using both avoids either missing slow degradation or paging on end for minor blips.

☐ What should happen operationally when an error budget is exhausted?

Ideally there's a pre-agreed policy: feature releases pause or require extra sign-off, and the team's priority shifts to reliability work until the budget recovers. Without a pre-agreed policy, this becomes a political argument in the moment — so part of implementing SLOs well is getting that policy agreed before you need it.

☐ How do internal SLOs typically relate to external, contractual SLAs?

The SLO should be stricter than the SLA — e.g. an internal 99.95% SLO backing a contractual 99.9% SLA — so that normal operational noise doesn't turn into a contractual breach and credit payout. The SLA is the customer-facing floor; the SLO is the internal ceiling you actually manage against.

☐ How do you decide whether a service needs a 99.9% vs 99.99% SLO?

Base it on actual business/user impact and cost, not aspiration — every additional 9 gets exponentially more expensive to engineer for. Ask what the real cost of downtime is (revenue, safety, regulatory exposure) and whether the current architecture can even support the target without a fundamental redesign, then set the target to match reality rather than picking a round number.

☐ Explain multi-window, multi-burn-rate alerting.

Instead of one static threshold, you alert on burn rate across multiple time windows simultaneously (e.g. a short 5-minute window catching sudden severe spikes, and a longer 1-hour window catching sustained moderate degradation) — this reduces both false positives from brief blips and false negatives from slow, steady leaks.

☐ How would you explain an SLO breach to a non-technical stakeholder?

Translate the number into business impact rather than leading with the metric — e.g. 'roughly 1 in 500 customer requests failed during this window, concentrated around the deployment at 2pm' rather than 'p99 latency exceeded 800ms.' Then state what's being done about it and by when, since stakeholders generally want to know impact and next steps more than the raw metric.

☐ How do the golden signals (latency, traffic, errors, saturation) relate to SLIs?

The golden signals are the standard starting menu for picking SLIs — most services can build a solid SLI set just from latency (how fast), traffic (how much load), errors (how often it fails), and saturation (how close to capacity), rather than inventing metrics from scratch.

AWS Solutions

CORE SERVICES

☐Walk through basic VPC design — subnets, route tables, NAT vs Internet Gateway.

A VPC is split into subnets, typically public (route to an Internet Gateway, for load balancers/bastion hosts) and private (route outbound traffic through a NAT Gateway, no direct inbound internet access — for application servers and databases). Route tables control which subnet uses which gateway; this separation keeps backend resources unreachable directly from the internet.

☐What are IAM best practices you follow?

Least privilege by default (start with nothing, add only what's needed), prefer roles over long-lived user access keys — especially for workloads (EC2/Lambda/ECS should assume a role, not embed static keys), use groups to manage human user permissions rather than per-user policies, and enable MFA for anything with elevated access. Understand policy evaluation logic: an explicit Deny always wins over an Allow.

☐RDS vs Aurora — when would you choose one over the other, and how does failover differ?

RDS runs standard engines (MySQL/Postgres/etc.) with traditional replication; Aurora is AWS's cloud-native engine with storage decoupled from compute across multiple AZs, giving faster failover (often under 30 seconds) because a replica is promoted against the same shared storage layer rather than needing to catch up first. Choose Aurora for MySQL/Postgres-compatible workloads needing higher availability and read-scaling; choose standard RDS for engines Aurora doesn't support or simpler/cheaper workloads.

☐What are S3 storage classes and how do lifecycle policies work?

Classes range from Standard (frequent access) through Infrequent Access, Glacier, and Glacier Deep Archive (cheaper storage, higher retrieval cost/latency). Lifecycle policies automatically transition objects between classes or expire them based on age — e.g. move logs to Infrequent Access after 30 days and Glacier after 90, without manual intervention.

☐What's a common Auto Scaling Group pitfall around health checks?

If the health-check grace period is too short relative to how long the application actually takes to become ready, the ASG can mark a perfectly healthy but still-starting instance as unhealthy and cycle it repeatedly — masking the real issue and causing constant churn. Always align the grace period with real observed startup time.

☐Compare ALB, NLB, and Classic Load Balancer.

ALB (Application Load Balancer) operates at Layer 7 — supports path/host-based routing, ideal for HTTP(S) microservices. NLB (Network Load Balancer) operates at Layer 4 — ultra-low latency, static IPs, handles millions of requests/sec, used for TCP/UDP or extreme performance needs. Classic Load Balancer is the legacy option, largely superseded by the other two.

☐What causes Lambda cold starts, and how do you mitigate them?

A cold start happens when Lambda has to initialize a new execution environment (download code, start the runtime, run init code) rather than reuse a warm one. Mitigate with provisioned concurrency for latency-sensitive functions, keeping deployment packages small, minimizing heavy work in global/init scope, and choosing a faster-starting runtime where possible.

☐ECS vs EKS — how do you decide which to use?

ECS is AWS-native, simpler to operate, and sufficient if you don't need Kubernetes-specific portability or its ecosystem (Helm, custom operators, multi-cloud consistency). EKS is the right call when you need Kubernetes itself — either for portability across clouds, an existing K8s-based toolchain, or because the team's expertise and tooling are already K8s-centric.

☐How do you design CloudWatch alerting to avoid noise?

Alert on symptoms that matter to users (error rate, latency, saturation) rather than every possible metric, use composite alarms to require multiple signals before paging, and use anomaly-detection alarms for metrics with natural variability (like traffic) instead of static thresholds that trigger false positives on predictable daily patterns.

☐How do you handle cross-account access in AWS?

Use IAM roles with a trust policy allowing the other account to assume the role (sts:AssumeRole), rather than sharing credentials — this gives auditable, time-limited access and avoids duplicating IAM users across accounts. Resource-based policies (e.g. an S3 bucket policy) are the alternative when the resource itself needs to grant cross-account access directly.

☐Can an RDS instance or an S3 bucket be renamed?

RDS instances CAN be renamed (modify-db-instance --new-db-instance-identifier) but the endpoint/DNS name changes and a reboot occurs, so downstream configs need updating. S3 buckets CANNOT be renamed at all — bucket names are globally unique with no rename API, so a 'rename' actually means creating a new bucket and copying/migrating objects. This distinction trips up a lot of engineers under interview pressure — good to have memorized cold.

☐How do you balance cost and reliability in an AWS architecture?

Use Reserved Instances or Savings Plans for steady-state, predictable baseline load; use Spot for stateless/fault-tolerant batch or non-critical workloads where interruption is acceptable; keep the reliability-critical path (databases, core routing) on stable on-demand/reserved capacity rather than chasing savings where it would compromise availability.

Kubernetes

TROUBLESHOOTING & CONCEPTS

☐A pod is stuck in Pending — how do you diagnose it?

kubectl describe pod shows the Events section, which almost always states the reason directly — most commonly insufficient CPU/memory on any node, an unsatisfiable node selector/affinity rule, or no available PersistentVolume matching a claim. Start there before digging deeper.

☐A pod is in CrashLoopBackOff — what's your diagnosis sequence?

kubectl logs <pod> (and --previous to see the log from before the last crash, since the current container may not have logged anything yet) to find the application-level error, then kubectl describe pod for OOMKilled or liveness-probe-triggered restarts, which look identical to an app crash from the surface but have a completely different fix.

☐A Service has no endpoints — what usually causes this?

Almost always a label selector mismatch between the Service and the pods it's supposed to target — the Service's selector doesn't match the pods' actual labels, or the pods aren't in a Ready state yet (a Running pod that's failing its readiness probe won't appear as an endpoint).

☐Deployment vs StatefulSet — when do you use each?

Deployment is for stateless, interchangeable replicas — any pod can be replaced by any other with no identity requirement. StatefulSet is for workloads needing stable network identity, stable storage per replica, and ordered startup/shutdown — like databases or anything where 'which specific instance' matters.

☐Explain requests vs limits, and what QoS class results.

Requests are what's guaranteed/reserved for scheduling; limits are the hard ceiling. A pod with requests==limits for both CPU and memory gets Guaranteed QoS (last to be evicted under pressure); requests less than limits gives Burstable; no requests/limits at all gives BestEffort (first to be evicted).

☐Compare HPA, VPA, and Cluster Autoscaler.

HPA (Horizontal Pod Autoscaler) adds/removes pod replicas based on metrics like CPU or custom metrics. VPA (Vertical Pod Autoscaler) adjusts a pod's own requests/limits over time. Cluster Autoscaler adds/removes nodes when pods can't be scheduled due to insufficient cluster capacity, or removes underutilized nodes. They solve different layers and are often used together.

☐A PodDisruptionBudget is blocking a node drain — how do you handle it?

First understand why the PDB exists (usually to guarantee minimum availability during voluntary disruptions) before overriding it. If the drain is genuinely necessary and safe, you can temporarily adjust the PDB or force the eviction, but the better first step is checking whether the blocked pods can tolerate the disruption (e.g. is there actually enough capacity for a safe rolling replacement) rather than just forcing through.

☐You updated a ConfigMap but the running pod isn't picking up the change — why?

Kubernetes doesn't automatically restart pods when a ConfigMap changes — if it's mounted as a volume, the file content does eventually update in the container's filesystem (with some propagation delay), but the application itself typically doesn't re-read it without being restarted or explicitly designed to watch for file changes. If it's consumed as an environment variable, it genuinely never updates without a pod restart, since env vars are set once at container start.

☐A node shows NotReady — what do you check?

kubectl describe node for conditions (MemoryPressure, DiskPressure, network unavailable), then check kubelet health directly on the node (systemctl status kubelet, journalctl -u kubelet) — common causes are kubelet crashing, disk pressure from log/image buildup, or a network partition between the node and control plane.

☐How do you troubleshoot DNS/networking issues inside a cluster?

Check CoreDNS pod health and logs first (a common root cause), test resolution from inside a debug pod (nslookup against the service name), and check NetworkPolicies that might be silently blocking traffic between namespaces even though DNS resolves fine — a policy issue looks like 'connection refused/timeout' rather than a DNS failure, which helps distinguish the two.

☐Explain RBAC basics — Role vs ClusterRole.

A Role grants permissions within a single namespace; a ClusterRole grants permissions cluster-wide (or can be bound within a specific namespace via a RoleBinding referencing it). RoleBinding/ClusterRoleBinding actually attach the permissions to a user, group, or service account — the Role/ClusterRole alone defines the permissions but grants nothing until bound.

☐What EKS-specific considerations come up beyond vanilla Kubernetes?

IRSA (IAM Roles for Service Accounts) for giving pods fine-grained AWS permissions instead of node-wide IAM roles, managed node groups vs Fargate profiles for compute, and enabling control-plane logging (API server, audit, authenticator logs) to CloudWatch, which isn't on by default and is often the first thing missing when you need to debug an auth or scheduling issue after the fact.

Networking

FUNDAMENTALS

Why is the OSI model still relevant when troubleshooting?

It gives you a systematic order to isolate a problem — physical/link (is the cable/interface up), network (routing/IP reachability), transport (can the port even be opened), and application (is the service actually responding correctly). Working layer by layer from bottom up prevents wasting time debugging application code when the real issue is a security group at layer 3/4.

Quick subnetting example: how many usable hosts in a /26?

/26 gives 2^(32-26) = 64 total addresses, minus network and broadcast addresses = 62 usable hosts. Being fluent in quick CIDR math (/24=254 usable, /28=14 usable) is a common quick-fire interview check.

What are the main DNS record types and what do they do?

A (hostname to IPv4), AAAA (hostname to IPv6), CNAME (alias to another hostname), MX (mail server priority), TXT (arbitrary text, often used for domain verification/SPF), and NS (which nameservers are authoritative for the domain).

Walk through the TCP three-way handshake and a common connection issue tied to it.

SYN (client requests connection) → SYN-ACK (server acknowledges and responds) → ACK (client confirms) — connection is then established. A common issue is a SYN flood or an overloaded server not completing the handshake in time, seen as connection timeouts even though the server process itself is technically running; another is a firewall silently dropping SYN packets, which looks identical to the server being down from the client's perspective.

Someone reports 'I can't reach the service' — walk through your layer-by-layer approach.

DNS resolution first (dig/nslookup) → basic reachability (ping, keeping in mind ICMP may be blocked even when the host is fine) → can the specific port be opened (nc -zv host port) → if the port opens, is the application actually responding correctly (curl with verbose output, checking status code and response body) → only then look at application logs.

What's a common load balancer health-check misconfiguration?

Checking a path that doesn't reflect real application health (e.g. a static file that always returns 200 regardless of whether the app's dependencies, like its database connection, are actually working) — this makes the load balancer send traffic to instances that are 'up' but functionally broken.

VPC Peering vs Transit Gateway — when do you use each?

VPC Peering is simple, direct point-to-point connectivity between two VPCs but doesn't scale well (no transitive routing — connecting many VPCs means many individual peering connections). Transit Gateway acts as a central hub, letting many VPCs (and on-prem via VPN/Direct Connect) route through one place, and is the standard choice once you're beyond a handful of VPCs.

Security Group vs Network ACL — what's the practical difference?

Security Groups are stateful (return traffic is automatically allowed) and operate at the instance/ENI level, allow-rules only. NACLs are stateless (you must explicitly allow both inbound and outbound), operate at the subnet level, and support both allow and deny rules — useful for explicitly blocking a known-bad IP range at the subnet boundary rather than in every security group.

Security

PRACTICES & INCIDENT RESPONSE

☐How do you apply the principle of least privilege in practice?

Start every new role/policy with zero permissions and add only what's proven necessary through actual usage (e.g. via IAM Access Analyzer/CloudTrail-informed policy generation), scope permissions to specific resources/actions rather than wildcard *, and review/prune unused permissions periodically rather than only adding, never removing.

☐What secrets management approaches do you use?

AWS Secrets Manager or Parameter Store (SecureString) for AWS-native workloads with automatic rotation support, HashiCorp Vault for more complex/multi-cloud secret needs, and KMS for envelope encryption underneath any of these. The common thread: secrets are never stored in code, config files committed to git, or environment variables baked into an image.

☐How do you approach patch management across a fleet?

Automate where possible (AWS Systems Manager Patch Manager or equivalent), patch in waves/canary groups rather than the whole fleet at once so a bad patch doesn't take everything down simultaneously, and track compliance/patch status centrally so gaps are visible rather than assumed.

☐Walk through your first steps on a suspected security incident.

Contain first, investigate second — isolate the affected resource (e.g. quarantine security group, disable compromised credentials) without necessarily destroying evidence, preserve logs, and notify the appropriate team/escalation path per the incident response plan. Avoid the instinct to immediately terminate/rebuild before understanding scope, since that can destroy forensic evidence needed to know what else was touched.

☐How do you implement encryption at rest and in transit?

At rest: enable encryption on storage services (EBS, S3 SSE, RDS storage encryption) using KMS-managed keys. In transit: enforce TLS everywhere (ALB listeners, RDS SSL connections, internal service-to-service mTLS where warranted) and reject plaintext connections where the service supports enforcing it.

☐Why are IAM roles generally preferred over IAM users for workloads?

Roles provide temporary, automatically-rotated credentials assumed at runtime rather than long-lived static access keys that can leak and remain valid indefinitely. A compromised role credential expires quickly; a leaked static access key is a standing risk until someone notices and rotates it.

☐How do you approach container image security?

Scan images for known CVEs in the build pipeline (e.g. Trivy) before they're allowed to deploy, use minimal base images to reduce attack surface, pin dependency versions rather than floating tags, and avoid running containers as root by default.

☐Given KPMG's audit/regulated-client context, what compliance awareness is relevant here?

Familiarity with frameworks like SOC 2 and ISO 27001 — change management with audit trails, access reviews, and evidence of least-privilege enforcement — is valuable in a Big Four environment since client engagements are often themselves audit-adjacent. Bring your own: reference your Barclays/regulated-banking experience as direct evidence you're used to operating under this level of scrutiny.

Storage

TYPES & DESIGN

☐ Block, object, and file storage — when do you use each?

Block storage (EBS) attaches to a single instance like a raw disk, needed for databases and low-latency filesystem access. Object storage (S3) is for unstructured data accessed over HTTP APIs at massive scale — backups, static assets, logs. File storage (EFS/FSx) provides a shared, POSIX-compliant filesystem multiple instances can mount simultaneously.

☐ EBS volume types — gp3 vs io2, when would you choose each?

gp3 is the general-purpose default for most workloads, with baseline IOPS/throughput independently configurable from volume size (an improvement over gp2). io2 is for high-performance, IOPS-intensive workloads (large databases) where you need very high, consistent IOPS and stronger durability guarantees than gp3 provides.

☐ What's S3's durability and availability model?

S3 Standard is designed for 99.999999999% (11 nines) durability by redundantly storing objects across multiple AZs, with 99.99% availability SLA. It offers strong read-after-write consistency for all operations, meaning a read immediately after a write reflects that write.

☐ How do you design a backup strategy?

Start from business requirements, not technology — define RPO (acceptable data loss window) and RTO (acceptable recovery time) first, then choose backup frequency, retention, and storage location (including cross-region for disaster scenarios) to meet those targets, and actually test restores periodically rather than assuming backups work.

☐ Explain PersistentVolumes/PersistentVolumeClaims in Kubernetes and reclaim policy.

A PV represents actual provisioned storage; a PVC is a pod's request for storage matching certain criteria (size, access mode, storage class), which Kubernetes binds to a matching PV. Reclaim policy determines what happens to the underlying storage when the PVC is deleted — Retain (keep the data, manual cleanup needed) vs Delete (storage is destroyed automatically) — getting this wrong on a stateful workload can mean either an unwanted data-loss risk or an accumulating pile of orphaned volumes.

☐ How do you handle storage capacity proactively rather than reactively?

Alert on a trend (e.g. 'will hit 90% in 7 days at current growth rate'), not just a static threshold, so there's lead time to act; combine with automated lifecycle policies (moving cold data to cheaper/archival tiers) so growth doesn't silently consume expensive storage.

Observability (Prometheus/Graphite/Grafana/ELK)

METRICS, DASHBOARDS & LOGS

☐Explain Prometheus's basic architecture.

Prometheus scrapes (pulls) metrics from targets on an interval via HTTP endpoints (either instrumented directly or via exporters like node_exporter), stores them in its own time-series database (TSDB), and Alertmanager (a separate component) handles routing/deduplicating/silencing alerts fired by Prometheus's rule evaluation.

☐What's the difference between rate() and irate() in PromQL, and when do you use each?

rate() calculates the average per-second rate over the whole time range given, smoothing out spikes — good for alerting and dashboards where you want stability. irate() uses only the last two data points, so it's more reactive to sudden spikes but noisier — better for fast-moving debugging graphs than for alert thresholds.

☐How does Alertmanager handle routing, grouping, and silencing?

Routing sends alerts to different receivers (Slack, PagerDuty, email) based on label matching (e.g. severity=critical goes to PagerDuty). Grouping batches related alerts into a single notification (e.g. all pods down in one deployment become one alert, not twenty) to reduce noise. Silencing temporarily suppresses alerts matching certain labels — used during planned maintenance so expected alerts don't page anyone.

☐What makes a good on-call Grafana dashboard?

Leads with the golden signals for the service at a glance, is organized top-down from 'is there a problem' to 'where is the problem' (overview panel first, drill-down panels below), and avoids dozens of panels someone has to hunt through at 3am — the goal is fast triage, not comprehensive data dumping.

☐How does Graphite differ from Prometheus in its data model?

Graphite uses a simpler dot-delimited metric-naming hierarchy (e.g. servers.web1.cpu.load) with limited built-in labeling, historically paired with a separate storage backend (whisper). Prometheus uses a multi-dimensional label model (metric_name{label=value,...}), which makes flexible querying and aggregation across dimensions much easier without needing to encode everything into the metric name itself.

☐Describe the ELK stack's components and how they fit together.

Elasticsearch stores and indexes log data for fast search; Logstash (or lighter-weight Beats agents) collect, parse, and ship logs into Elasticsearch; Kibana provides the visualization/search UI on top. Together they form a centralized logging pipeline so logs from many hosts/services are searchable in one place instead of SSH-ing into each host.

☐What is Index Lifecycle Management in Elasticsearch, and why does it matter?

ILM automatically moves indices through phases (hot → warm → cold → delete) based on age or size, moving older, less-frequently-queried log data to cheaper storage tiers and eventually deleting it per retention policy — without it, indices accumulate indefinitely and both cost and query performance degrade over time.

☐How do you use Kibana during an actual incident?

Build a quick filtered search scoped to the affected service and time window, then use visualizations (like a bar chart of error types over time) to spot when a specific error first appeared and whether it correlates with a deploy or config change, rather than scrolling raw logs manually.

☐How do you design alerting to reduce noise and alert fatigue?

Alert only on symptoms that require human action (not every anomaly), tune thresholds against real historical data rather than guesses, use multi-window burn-rate style alerting instead of single static thresholds, and route by actual severity so low-priority signals don't page anyone at night. This kind of systematic review is what drove a roughly 40% alert-noise reduction in one of my past roles — fewer, more meaningful alerts, not just quieter ones.

☐How do you correlate metrics, logs, and traces during an incident?

Start from the metric that triggered the alert to establish the time window and affected service, pull logs scoped to that exact window and service to find the specific error, and if traces are available, follow a slow/failed request end to end across services to see exactly where time was lost or an error originated — metrics tell you something's wrong, logs and traces tell you why.

☐What causes cardinality explosion in Prometheus, and how do you mitigate it?

Using high-cardinality values as label values (like a raw user ID or a full URL path with dynamic segments) causes Prometheus to create a huge number of unique time series, which can degrade performance or exhaust memory. Mitigate by avoiding unbounded labels, aggregating at the source before exposing metrics, or using recording rules to pre-aggregate before storage.

☐Push vs pull metrics models — how do you choose?

Pull (Prometheus's default model) works well for long-lived services with stable, discoverable endpoints, and keeps the scraper in control of load. Push is better for short-lived jobs (batch/cron tasks) that finish before a pull could happen, typically via a push gateway that holds the metric until the next scrape.

Behavioral & Interpersonal

STAR-STYLE FRAMEWORKS

- [ ] **Tell me about a time you resolved a major incident.**

  Bring your own real incident, structured as Situation → Task → Action → Result. You have verified material to anchor to: a chaos-engineering initiative that identified 3 single points of failure and contributed to a ~45% improvement in recovery time, or an alerting/observability overhaul that reduced alert noise by ~40% and improved detection speed. Pick the one closest to what KPMG's platform likely looks like and add the real specifics only you know.

- [ ] **Describe a conflict with a team member or stakeholder and how you resolved it.**

  Bring your own — structure as: what the disagreement was actually about (usually a tradeoff — speed vs safety, whose priority wins), how you approached the conversation (data/reasoning over authority), and the outcome. Avoid framing where you were simply 'right' and they were 'wrong'; interviewers are listening for how you handle disagreement, not who won.

- [ ] **How do you explain a technical problem to a non-technical stakeholder — relevant in a client-facing consulting environment like KPMG?**

  Lead with business impact, not mechanism ('checkout is failing for about 1 in 20 customers' before 'the payment service pod is CrashLoopBackOff'), avoid jargon unless asked to go deeper, and always pair the explanation with what's being done and a realistic timeline — stakeholders are usually most anxious about the unknown, not the technical detail itself.

- [ ] **Tell me about a time you improved a process through automation.**

  Bring your own, anchored to a real result — e.g. an automation initiative that reduced manual deployment effort by roughly 60%. Structure as: what was manual and error-prone → what you built and why that approach → the measured improvement → what you'd still improve further, which shows ongoing ownership rather than a one-and-done project.

- [ ] **How do you approach being on-call sustainably, avoiding burnout for yourself or your team?**

  Bring your own philosophy, but strong answers usually include: reducing noisy/actionless alerts so on-call time isn't spent triaging false positives, maintaining clear runbooks so incidents don't always require the same one or two people, and treating a rough on-call rotation as a signal to fix the underlying reliability gap, not just something to tough out.

- [ ] **Describe how you mentor or support less experienced team members.**

  Bring your own — concrete examples land better than general statements (pairing on an actual incident post-mortem, walking someone through a runbook rather than just doing the fix yourself, reviewing their IaC changes with explanation rather than just approving).

- [ ] **How do you stay current with new technologies and practices?**

  Bring your own routine, but tie it to something concrete and recent rather than a vague 'I read blogs' — e.g. hands-on exploration of a specific new AWS service, a cert you recently pursued (your SC-100/SC-300 track is good evidence here), or a tool you evaluated and either adopted or consciously decided against.

- [ ] **How do you balance compliance/regulatory requirements with the need to move fast, given your banking background?**

  Bring your own, but a strong framing: treat compliance requirements (change approval, audit trails, access reviews) as things you engineer around rather than fight — e.g. automating the evidence-collection/approval trail itself so it doesn't become a manual bottleneck. Reference your Barclays experience directly as proof you're comfortable operating this way already.